

Course Code: CSA1709

Course Name: Artificial Intelligence

Lab Manual

Ex.No:1

Experiment: Write a Python Program to Solve the 8-Puzzle Problem (Using Breadth First Search)

Aim

To develop a Python program that solves the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm and finds the shortest path from the initial state to the goal state.

Objectives

- To understand the concept of the 8-Puzzle problem.
- To implement the Breadth-First Search (BFS) algorithm in Python.
- To find the shortest sequence of moves from the initial state to the goal state.
- To display the solution path if it exists.
- To understand state-space search in Artificial Intelligence.

Algorithm

1. Start.
2. Read the initial state of the 8-puzzle.
3. Define the goal state.
4. Create a queue and insert the initial state.
5. Create a visited set to avoid revisiting states.
6. Remove the first state from the queue.
7. If the current state is the goal state, display the solution path and stop.
8. Otherwise, generate all possible next states by moving the blank tile (0) up, down, left, or right.
9. Add unvisited states to the queue and mark them as visited.
10. Repeat Steps 6–9 until the queue becomes empty.
11. If no solution is found, display "No Solution Exists."
12. Stop.

Program (Python)

```
from collections import deque
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)
moves = [(-1,0),(1,0),(0,-1),(0,1)]
def get_next_states(state):
    state = list(state)
```

```

zero = state.index(0)
row = zero // 3
col = zero % 3
next_states = []
for dr, dc in moves:
    nr = row + dr
    nc = col + dc
    if 0 <= nr < 3 and 0 <= nc < 3:
        new = state[:]
        new_zero = nr * 3 + nc
        new[zero], new[new_zero] = new[new_zero], new[zero]
        next_states.append(tuple(new))
return next_states
def bfs(start):
    queue = deque([(start, [])])
    visited = set()
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for next_state in get_next_states(state):
                queue.append((next_state, path + [state]))
    return None
start = (1, 2, 3,
        4, 5, 6,
        7, 0, 8)
solution = bfs(start)
if solution:
    print("Solution Found!\n")
    for step in solution:
        for i in range(0, 9, 3):
            print(step[i:i+3])
        print()
else:
    print("No Solution Exists.")

```

Sample Input

Initial State

1 2 3

4 5 6

7 0 8

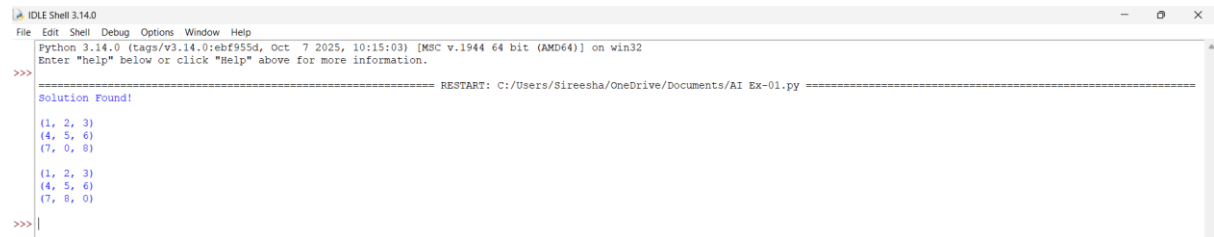
Goal State

1 2 3

4 5 6

7 8 0

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-01.py =====
Solution Found!
[(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)]
>>>
```

Result

The Python program was successfully implemented using the Breadth-First Search (BFS) algorithm to solve the 8-Puzzle problem. The program found the shortest sequence of moves from the initial state to the goal state and displayed the solution correctly.

Ex.No:2

Experiment: Write a Python Program to Solve the 8-Queen Problem

Aim

To develop a Python program that solves the 8-Queen problem using the Backtracking algorithm by placing eight queens on a chessboard such that no two queens attack each other.

Objectives

- To understand the 8-Queen problem.
- To implement the Backtracking algorithm in Python.
- To place eight queens safely on the chessboard.
- To ensure that no two queens share the same row, column, or diagonal.
- To display one valid solution.

Algorithm

1. Start.
2. Create an empty 8×8 chessboard.
3. Begin placing queens column by column.
4. For each column, check every row.
5. Verify whether placing a queen is safe by checking:
 - Left row
 - Upper-left diagonal
 - Lower-left diagonal
6. If the position is safe, place the queen.
7. Recursively place the queen in the next column.
8. If no safe position exists, remove the previously placed queen (backtracking).
9. Repeat until all eight queens are placed.
10. Display the board.
11. Stop.

Program (Python)

```
N = 8
```

```
# Function to print the board
```

```

def print_board(board):
    for row in board:
        print(row)

# Function to check if a queen can be placed
def is_safe(board, row, col):
    # Check left side of the row
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper-left diagonal
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True

# Backtracking function
def solve_queen(board, col):
    if col >= N:
        return True

```

```

for row in range(N):
    if is_safe(board, row, col):
        board[row][col] = 1
        if solve_queen(board, col + 1):
            return True
        board[row][col] = 0
    return False

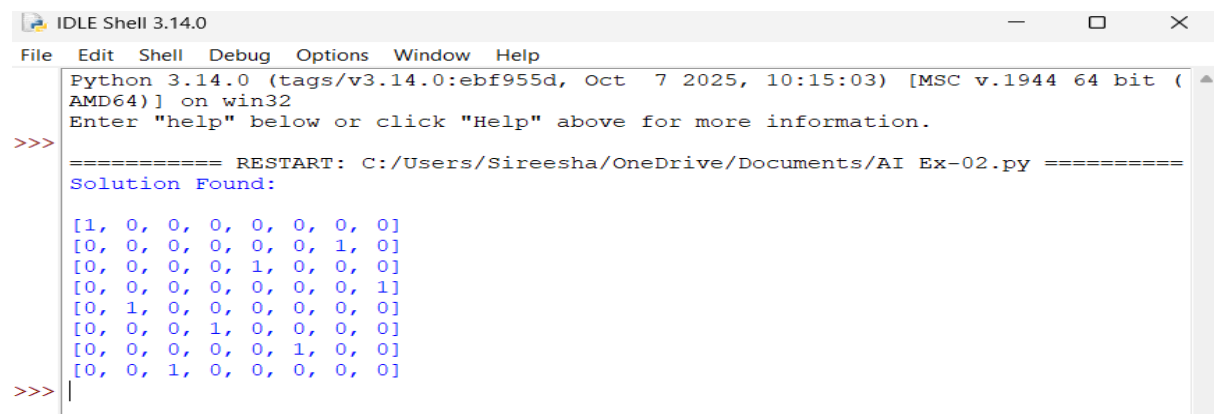
board = [[0 for _ in range(N)] for _ in range(N)]
if solve_queen(board, 0):
    print("Solution Found:\n")
    print_board(board)
else:
    print("No Solution Exists")

```

Sample Input

N = 8

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-02.py =====
Solution Found:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
>>>

```

Result

The Python program was successfully implemented using the Backtracking algorithm to solve the 8-Queen problem. The program placed all eight queens on the chessboard such that no two queens attacked each other and displayed a valid solution.

Ex.No:3

Experiment : Write a Python Program for the Water Jug Problem

Aim

The aim of this experiment is to write a Python program to solve the Water Jug Problem and obtain the required amount of water using two jugs by performing fill, empty, and transfer operations.

Objectives

- To understand the Water Jug problem.
- To implement the solution using Python.
- To perform filling, emptying, and transferring operations.
- To display the sequence of steps to reach the goal.

Algorithm

1. Start.
2. Initialize two jugs with their capacities.
3. Fill the first jug.
4. Pour water from the first jug to the second jug.
5. Empty the second jug when it becomes full.
6. Repeat the process until the required amount of water is obtained.
7. Display all the steps.
8. Stop.

Program (Python)

```
def water_jug(jug1, jug2, target):  
    x = 0  
    y = 0  
    print("Steps:")  
    while (x != target) and (y != target):  
        if x == 0:  
            x = jug1  
        print(x, y)
```

```

elif y == jug2:
    y = 0
    print(x, y)
else:
    transfer = min(x, jug2 - y)
    x -= transfer
    y += transfer
    print(x, y)

print("\nTarget Amount Reached!")

# Capacity of jugs: 4L and 3L
water_jug(4, 3, 2)

```

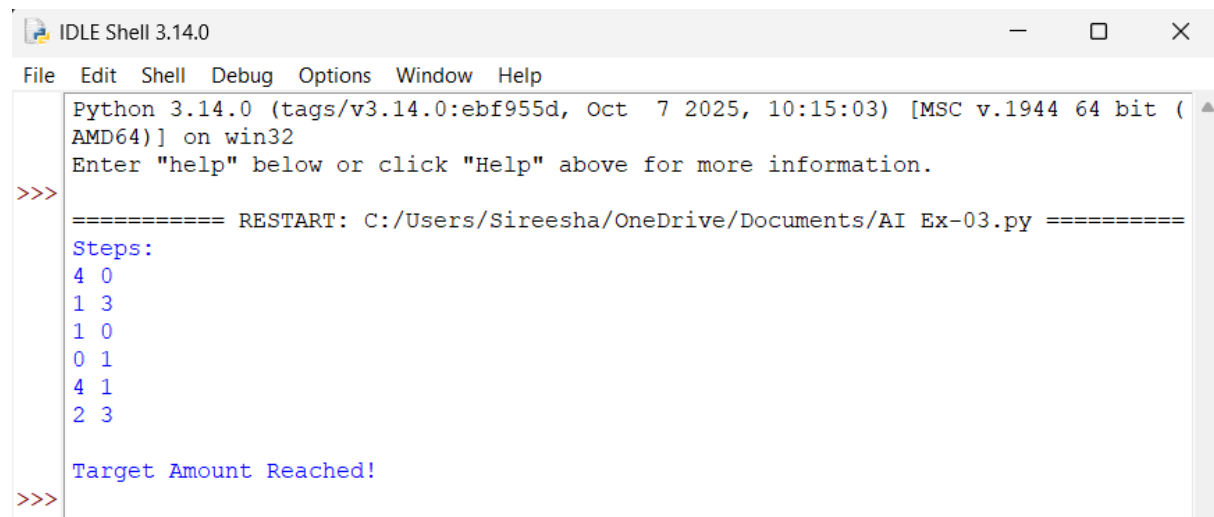
Sample Input

Jug 1 Capacity = 4 Litres

Jug 2 Capacity = 3 Litres

Target = 2 Litres

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-03.py =====
Steps:
4 0
1 3
1 0
0 1
4 1
2 3
Target Amount Reached!
>>>

```

Result

The Python program for the Water Jug Problem was successfully implemented. The program performed filling, emptying, and transferring operations to obtain the required amount of water (2 litres) and displayed the sequence of steps correctly.

Ex.No :4

Experiment : Write a Python Program for the Crypt-Arithmetic Problem

Aim

The aim of this experiment is to write a Python program to solve the Crypt-Arithmetic problem by assigning digits to letters such that the given arithmetic equation is satisfied.

Objectives

- To understand the Crypt-Arithmetic problem.
- To write a Python program to solve it.
- To assign unique digits to letters.
- To display the correct solution.

Algorithm

1. Start.
2. Read the given Crypt-Arithmetic expression.
3. Identify all unique letters.
4. Assign different digits (0–9) to each letter.
5. Check whether the arithmetic equation is satisfied.
6. If the equation is correct, display the solution.
7. Otherwise, try another combination.
8. Stop.

Program(Python)

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

for p in permutations(range(10), 8):

    d = dict(zip(letters, p))

    # First letters cannot be zero
    if d['S'] == 0 or d['M'] == 0:

        continue

    send = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']

    more = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
```

```

money = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']
if send + more == money:
    print("Solution Found\n")
    print("SEND =", send)
    print("MORE =", more)
    print("MONEY =", money)
    print("\nLetter Values:")
    for k, v in d.items():
        print(k, "=", v)
    break

```

Sample Input

SEND

+ MORE

MONEY

Output

```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-04.py =====
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Values:
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2
>>>

```

Result

The Python program for the Crypt-Arithmetic Problem was successfully implemented. The program assigned unique digits to each letter and correctly solved the equation SEND + MORE = MONEY.

Ex.No :5

Experiment : Write a Python Program for the Missionaries and Cannibals Problem

Aim

The aim of this experiment is to write a Python program to solve the Missionaries and Cannibals problem by safely transporting all missionaries and cannibals across the river without violating the given conditions.

Objectives

- To understand the Missionaries and Cannibals problem.
- To write a Python program to solve the problem.
- To move all missionaries and cannibals safely across the river.
- To display the sequence of valid moves.

Algorithm

1. Start.
2. Initialize the number of missionaries and cannibals on both sides of the river.
3. Place the boat on the left side.
4. Generate all valid boat moves.
5. Check that missionaries are never outnumbered by cannibals on either side.
6. Move the boat and update the state.
7. Repeat until everyone reaches the right side.
8. Display the solution path.
9. Stop.

Program (Python)

```
from collections import deque

def is_valid(m_left, c_left):
    m_right = 3 - m_left
    c_right = 3 - c_left
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False
```

```

if (m_left > 0 and c_left > m_left):
    return False

if (m_right > 0 and c_right > m_right):
    return False

return True

def solve():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')
    moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
    queue = deque([(start, [start])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path

        if state in visited:
            continue

        visited.add(state)

        m, c, boat = state

        for dm, dc in moves:
            if boat == 'L':
                new = (m-dm, c-dc, 'R')
            else:
                new = (m+dm, c+dc, 'L')

            if is_valid(new[0], new[1]):
                queue.append((new, path + [new]))

    return None

solution = solve()

print("Solution Path:\n")

```

for step in solution:

print(step)

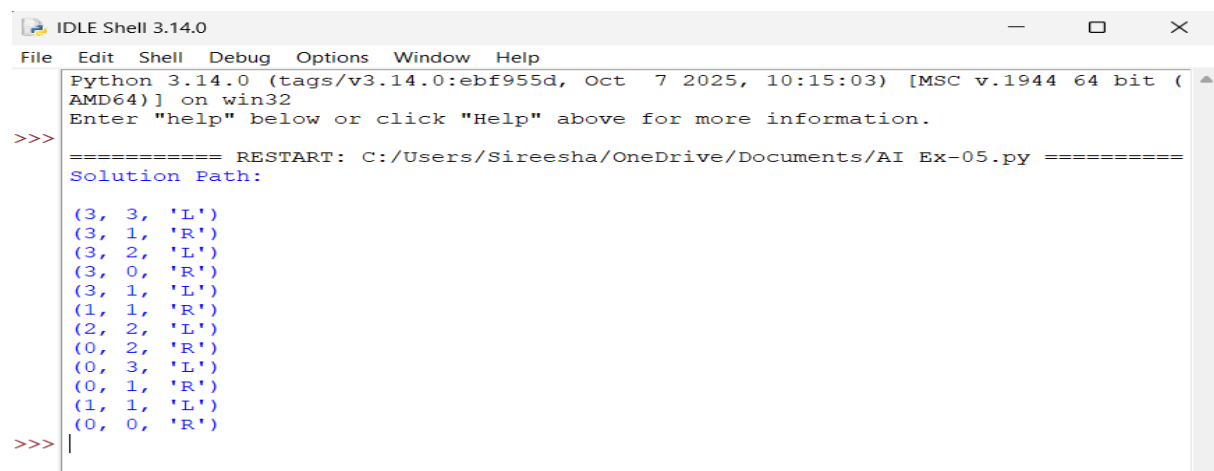
Sample Input

Missionaries = 3

Cannibals = 3

Boat Capacity = 2

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-05.py =====
Solution Path:
(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
>>> |
```

Result

The Python program for the Missionaries and Cannibals Problem was successfully implemented. The program safely transported all 3 missionaries and 3 cannibals across the river while ensuring that the number of cannibals never exceeded the number of missionaries on either side. The sequence of valid moves was displayed successfully.

Ex.No :6

Experiment : Write a Python Program for the Vacuum Cleaner Problem

Aim

The aim of this experiment is to write a Python program to simulate the Vacuum Cleaner problem, where the vacuum cleaner cleans all dirty rooms by moving between them.

Objectives

- To understand the Vacuum Cleaner problem.
- To write a Python program to simulate the vacuum cleaner.
- To clean all dirty rooms.
- To display the actions performed by the vacuum cleaner.

Algorithm

1. Start.
2. Initialize the rooms and their status (Clean/Dirty).
3. Place the vacuum cleaner in the starting room.
4. Check whether the current room is dirty.
5. If the room is dirty, clean it.
6. Move to the next room.
7. Repeat until all rooms are clean.
8. Display the actions and final room status.
9. Stop.

Program (Python)

```
# Vacuum Cleaner Problem
```

```
rooms = ['Dirty', 'Dirty']
```

```
location = 0
```

```
while location < len(rooms):
```

```
    if rooms[location] == 'Dirty':
```

```
        print("Room", location + 1, "is Dirty")
```

```
        print("Cleaning Room", location + 1)
```

```
        rooms[location] = 'Clean'
```

```

else:

    print("Room", location + 1, "is already Clean")

    location += 1

print("\nFinal Status of Rooms:")

for i in range(len(rooms)):

    print("Room", i + 1, ":", rooms[i])

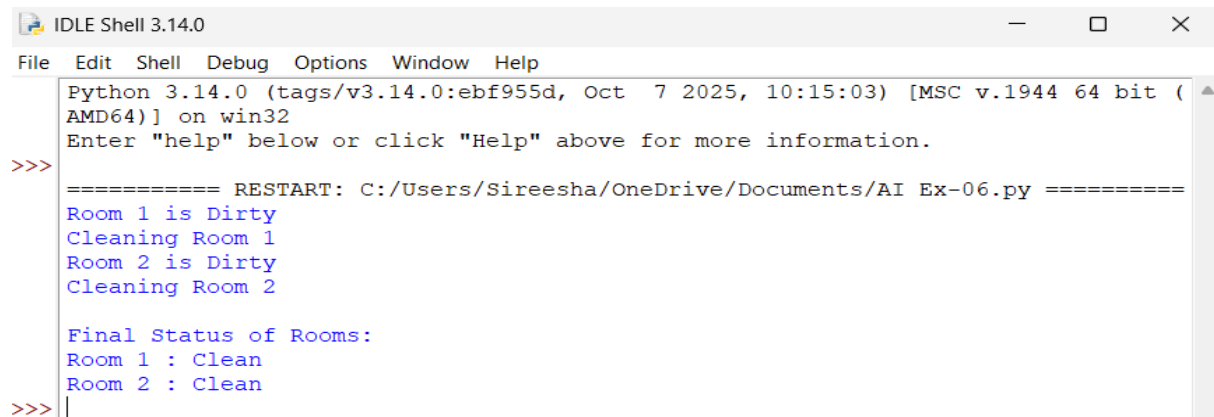
```

Sample Input

Room 1 = Dirty

Room 2 = Dirty

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-06.py =====
Room 1 is Dirty
Cleaning Room 1
Room 2 is Dirty
Cleaning Room 2

Final Status of Rooms:
Room 1 : Clean
Room 2 : Clean
>>>|

```

Result

The Python program for the Vacuum Cleaner Problem was successfully implemented. The vacuum cleaner cleaned all the dirty rooms and displayed the final status of each room as Clean.

Ex.No :7

Experiment : Write a Python Program to Implement Breadth-First Search (BFS)

Aim

The aim of this experiment is to write a Python program to implement the Breadth-First Search (BFS) algorithm to traverse all the vertices of a graph.

Objectives

- To understand the BFS algorithm.
- To write a Python program for BFS.
- To traverse all vertices of a graph.
- To display the BFS traversal order.

Algorithm

1. Start.
2. Create a graph using an adjacency list.
3. Select the starting vertex.
4. Add the starting vertex to the queue.
5. Remove a vertex from the queue and display it.
6. Visit all unvisited adjacent vertices and add them to the queue.
7. Repeat until the queue is empty.
8. Stop.

Program (Python)

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],
```



```
'D': [],  
'E': ['F'],  
'F': []  
}  
visited = set()  
queue = deque()  
start = 'A'  
visited.add(start)  
queue.append(start)  
print("BFS Traversal:")  
while queue:  
    vertex = queue.popleft()  
    print(vertex, end=" ")  
    for neighbor in graph[vertex]:  
        if neighbor not in visited:  
            visited.add(neighbor)  
            queue.append(neighbor)
```

Sample Input

Graph:

$A \rightarrow B, C$

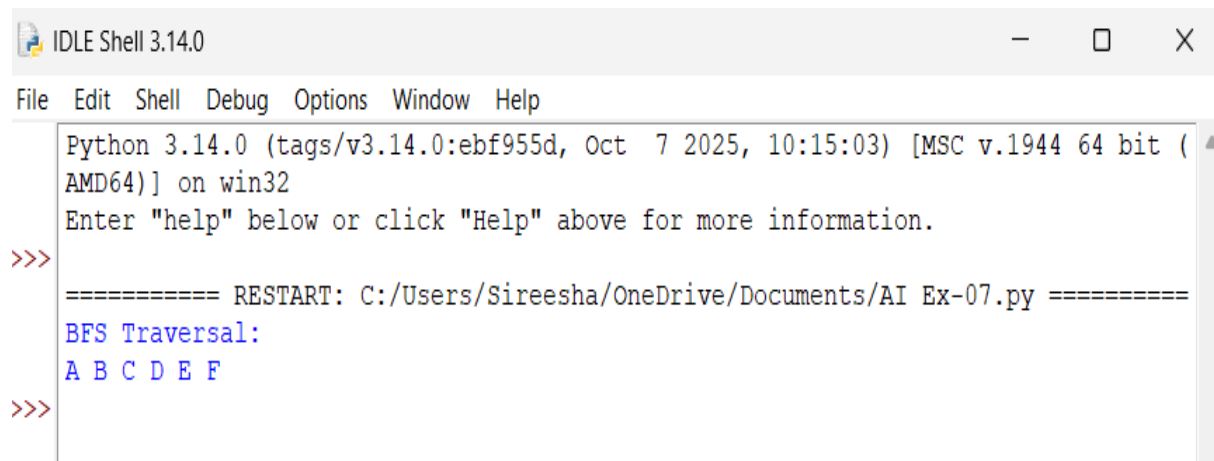
$B \rightarrow D, E$

$C \rightarrow F$

$E \rightarrow F$

Starting Vertex: A

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-07.py =====
BFS Traversal:
A B C D E F
>>>
```

Result

The Python program to implement the Breadth-First Search (BFS) algorithm was successfully executed. The graph was traversed level by level, and the BFS traversal order was displayed correctly.

Ex.No:8

Experiment : Write a Python Program to Implement Depth-First Search (DFS)

Aim

The aim of this experiment is to write a Python program to implement the Depth-First Search (DFS) algorithm to traverse all the vertices of a graph.

Objectives

- To understand the DFS algorithm.
- To write a Python program for DFS.
- To traverse all vertices of a graph.
- To display the DFS traversal order.

Algorithm

1. Start.
2. Create a graph using an adjacency list.
3. Select the starting vertex.
4. Mark the starting vertex as visited.
5. Display the current vertex.
6. Visit each unvisited adjacent vertex recursively.
7. Repeat until all vertices are visited.
8. Stop.

Program (Python)

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```

visited = set()

def dfs(vertex):
    if vertex not in visited:
        print(vertex, end=" ")
        visited.add(vertex)
        for neighbor in graph[vertex]:
            dfs(neighbor)

print("DFS Traversal:")
dfs('A')

```

Sample Input

Graph:

$A \rightarrow B, C$

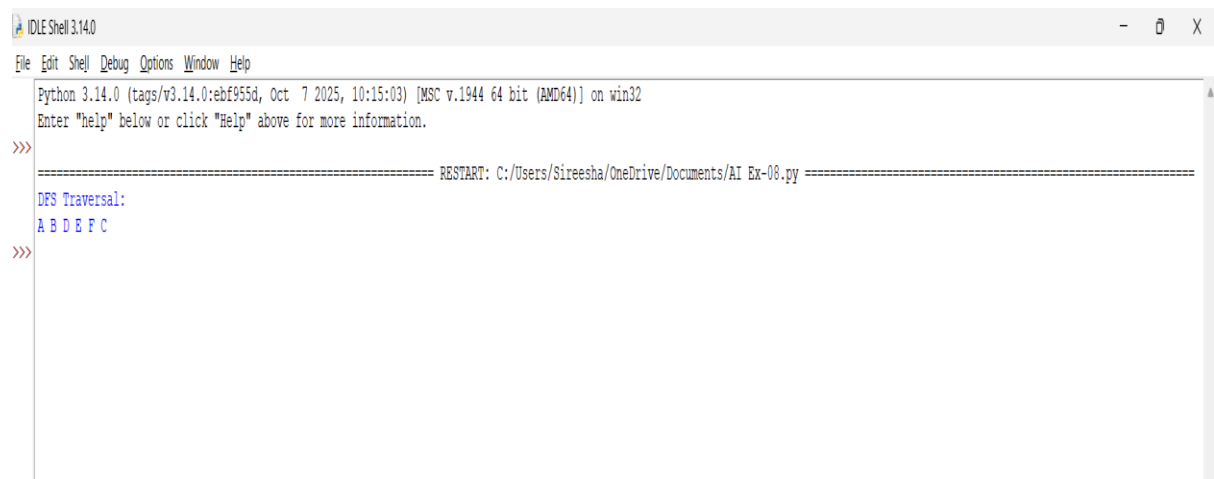
$B \rightarrow D, E$

$C \rightarrow F$

$E \rightarrow F$

Starting Vertex: A

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-08.py =====
DFS Traversal:
A B D E F C
>>>

```

Result

The Python program to implement the Depth-First Search (DFS) algorithm was successfully executed. The graph was traversed using the depth-first approach, and the DFS traversal order was displayed correctly.

Ex.No-9

Experiment : Write a Python Program to Implement the Travelling Salesman Problem (TSP)

Aim

The aim of this experiment is to write a Python program to solve the Travelling Salesman Problem by finding the shortest route that visits each city exactly once and returns to the starting city.

Objectives

- To understand the Travelling Salesman Problem (TSP).
- To write a Python program to solve TSP.
- To find the minimum travel cost.
- To display the shortest possible route.

Algorithm

1. Start.
2. Define the distance matrix between cities.
3. Select the starting city.
4. Generate all possible routes.
5. Calculate the total distance of each route.
6. Find the route with the minimum distance.
7. Display the shortest route and its cost.
8. Stop.

Program (Python)

```
from itertools import permutations
```

```
graph = [  
    [0, 10, 15, 20],  
    [10, 0, 35, 25],  
    [15, 35, 0, 30],  
    [20, 25, 30, 0]  
]
```

```

cities = [0, 1, 2, 3]
start = 0
min_cost = float('inf')
best_path = None
for path in permutations(cities[1:]):
    route = (start,) + path + (start,)
    cost = 0
    for i in range(len(route) - 1):
        cost += graph[route[i]][route[i + 1]]
    if cost < min_cost:
        min_cost = cost
        best_path = route
print("Shortest Route:", best_path)
print("Minimum Cost:", min_cost)

```

Sample Input

Distance Matrix

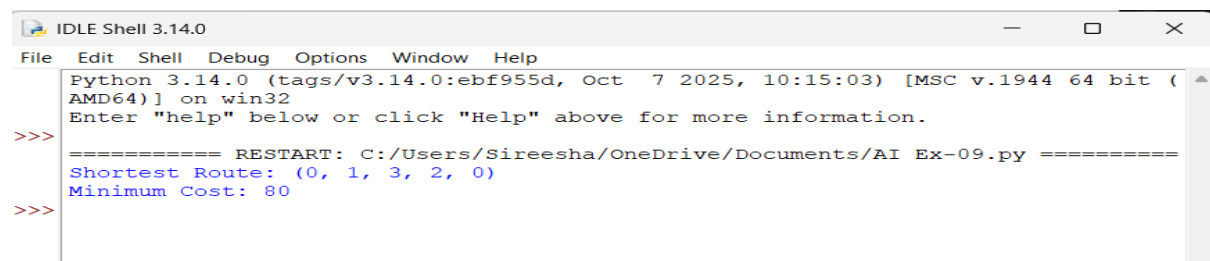
0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-09.py =====
Shortest Route: (0, 1, 3, 2, 0)
Minimum Cost: 80
>>>

```

Result

The Python program for the Travelling Salesman Problem (TSP) was successfully implemented. The program found the shortest route that visits all cities exactly once and returns to the starting city, and it displayed the minimum travel cost correctly.

Ex.No-10

Experiment 10: Write a Python Program to Implement the A Algorithm*

Aim

The aim of this experiment is to write a Python program to implement the A* algorithm to find the shortest path between a start node and a goal node.

Objectives

- To understand the A* search algorithm.
- To write a Python program for A*.
- To find the shortest path between two nodes.
- To display the shortest path.

Algorithm

1. Start.
2. Define the graph and heuristic values.
3. Select the start node and goal node.
4. Add the start node to the open list.
5. Select the node with the lowest cost.
6. Explore its neighboring nodes and update their costs.
7. Repeat until the goal node is reached.
8. Display the shortest path.
9. Stop.

Program (Python)

```
graph = {  
    'A': {'B': 1, 'C': 3},  
    'B': {'D': 3, 'E': 6},  
    'C': {'F': 5},  
    'D': {'G': 2},  
    'E': {'G': 1},  
    'F': {'G': 2},  
    'G': {}  
}
```

```

}
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 2,
    'G': 0
}

def a_star(start, goal):
    open_list = {start}
    closed_list = set()
    g = {start: 0}
    parent = {start: start}
    while open_list:
        node = min(open_list, key=lambda x: g[x] + heuristic[x])
        if node == goal:
            path = []
            while parent[node] != node:
                path.append(node)
                node = parent[node]
            path.append(start)
            path.reverse()
            return path
        open_list.remove(node)
        closed_list.add(node)
        for neighbor, cost in graph[node].items():
            if neighbor in closed_list:

```



```

        continue

    new_cost = g[node] + cost

    if neighbor not in open_list or new_cost < g.get(neighbor, float('inf')):

        g[neighbor] = new_cost

        parent[neighbor] = node

        open_list.add(neighbor)

    return None

path = a_star('A', 'G')

print("Shortest Path:", path)

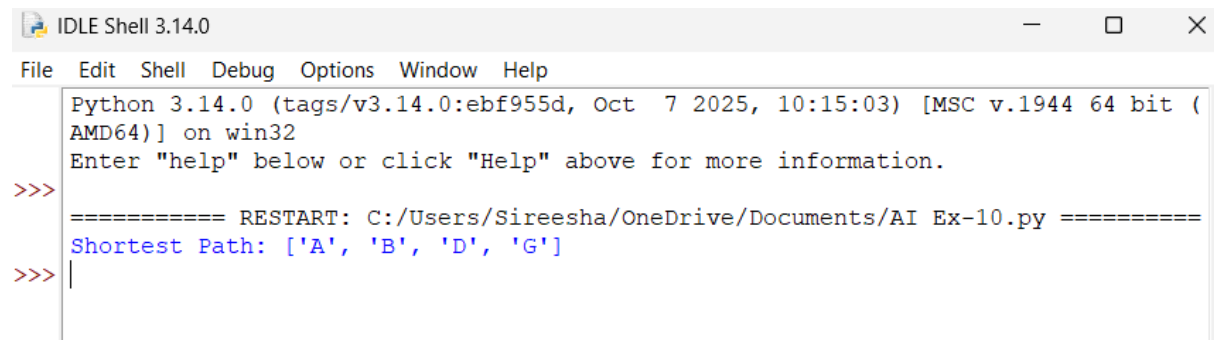
```

Sample Input

Start Node = A

Goal Node = G

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-10.py =====
Shortest Path: ['A', 'B', 'D', 'G']
>>>
|

```

Result

The Python program for the A* (A-Star) Algorithm was successfully implemented. The program found the shortest path from the start node to the goal node using the heuristic function and displayed the optimal path correctly.

Ex.No-11

Experiment : Write a Python Program for Map Coloring to Implement CSP

Aim

The aim of this experiment is to write a Python program to solve the Map Coloring problem using Constraint Satisfaction Problem (CSP) techniques by assigning different colors to adjacent regions.

Objectives

- To understand the Map Coloring problem.
- To implement CSP using Python.
- To assign different colors to adjacent regions.
- To display the valid color assignment.

Algorithm

1. Start.
2. Define the map with its neighboring regions.
3. Define the available colors.
4. Assign a color to each region.
5. Check that no two adjacent regions have the same color.
6. If a conflict occurs, try another color.
7. Repeat until all regions are colored.
8. Display the color assigned to each region.
9. Stop.

Program (Python)

```
# Map Coloring using CSP (Backtracking)
```

```
regions = ['A', 'B', 'C', 'D']
```

```
neighbors = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['A', 'C', 'D'],
```

```
    'C': ['A', 'B', 'D'],
```

```
    'D': ['B', 'C']
```

```

}
colors = ['Red', 'Green', 'Blue']
assignment = {}
def is_safe(region, color):
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True
def solve(index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_safe(region, color):
            assignment[region] = color
            if solve(index + 1):
                return True
            del assignment[region]
    return False
if solve(0):
    print("Color Assignment:")
    for region in regions:
        print(region, "->", assignment[region])
else:
    print("No Solution Exists")

```

Sample Input

Regions: A, B, C, D

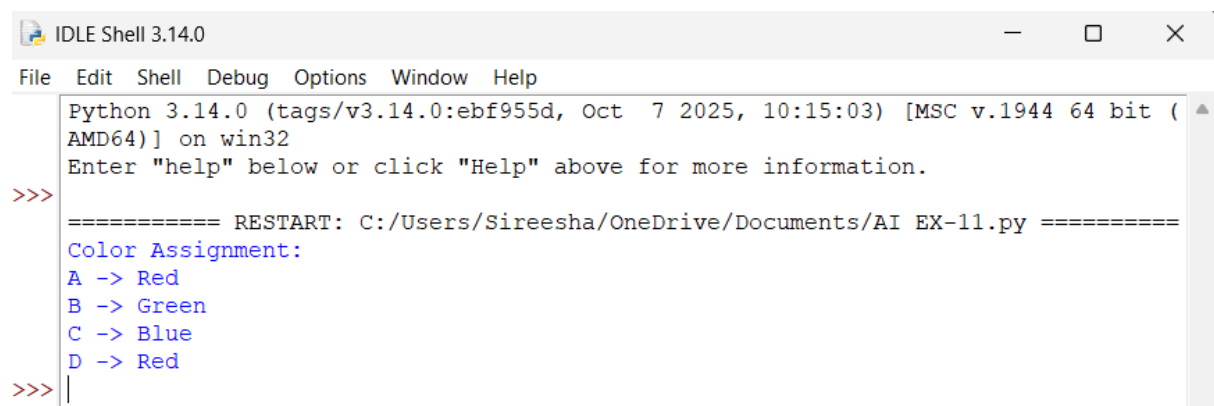
Colors:

Red

Green

Blue

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI EX-11.py =====
Color Assignment:
A -> Red
B -> Green
C -> Blue
D -> Red
>>> |
```

Result

The Python program for the Map Coloring Problem using Constraint Satisfaction Problem (CSP) was successfully implemented. The program assigned different colors to adjacent regions without any conflicts and displayed a valid color assignment.

Ex.No-12

Experiment : Write a Python Program for the Tic-Tac-Toe Game

Aim

The aim of this experiment is to write a Python program to implement the Tic-Tac-Toe game, where two players take turns marking **X** and **O** on a 3×3 board until one player wins or the game ends in a draw.

Objectives

- To understand the Tic-Tac-Toe game.
- To write a Python program for the game.
- To allow two players to play alternately.
- To determine the winner or declare a draw.

Algorithm

1. Start.
2. Create an empty 3×3 game board.
3. Allow Player X and Player O to take turns.
4. Accept the row and column as input.
5. Place the player's symbol in the selected position.
6. Check if any player has won.
7. If all cells are filled and no player wins, declare a draw.
8. Stop.

Program (Python)

```
board = [' ' for _ in range(9)]
```

```
def print_board():
```

```
    print(board[0], "|", board[1], "|", board[2])
```

```
    print("--+---+--")
```

```
    print(board[3], "|", board[4], "|", board[5])
```

```
    print("--+---+--")
```

```
    print(board[6], "|", board[7], "|", board[8])
```

```
def check_win(player):
```

```

wins = [
    [0,1,2], [3,4,5], [6,7,8],
    [0,3,6], [1,4,7], [2,5,8],
    [0,4,8], [2,4,6]
]

for w in wins:
    if board[w[0]] == board[w[1]] == board[w[2]] == player:
        return True
    return False

player = 'X'

for i in range(9):
    print_board()
    pos = int(input("Enter position (1-9): ")) - 1
    if board[pos] == ' ':
        board[pos] = player
    else:
        print("Position already filled!")
        continue
    if check_win(player):
        print_board()
        print(player, "wins!")
        break
    player = 'O' if player == 'X' else 'X'

else:
    print_board()
    print("Match Draw!")

```

Sample Input

Player X: 1

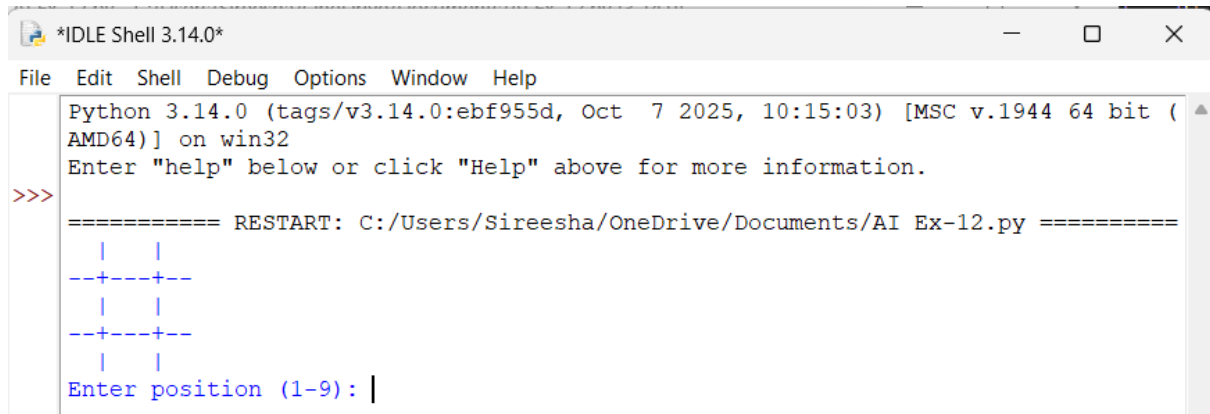
Player O: 5

Player X: 2

Player O: 8

Player X: 3

Sample Output



```
*IDLE Shell 3.14.0*
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-12.py =====
| | |
--+---+--
| | |
--+---+--
| | |
Enter position (1-9): |
```

Result

The Python program for the Tic-Tac-Toe Game was successfully implemented. The program allowed two players to play alternately, checked for a winner after each move, and correctly displayed the winner or declared the game as a draw.

Ex.No-13

Experiment 13: Write a Python Program to Implement the Minimax Algorithm for Gaming

Aim

The aim of this experiment is to write a Python program to implement the Minimax algorithm for a game and determine the best possible move for the player.

Objectives

- To understand the Minimax algorithm.
- To write a Python program for Minimax.
- To find the best move in a game.
- To display the optimal move.

Algorithm

1. Start.
2. Define the game state and possible moves.
3. Check if the game has reached a terminal state.
4. If it is the maximizing player's turn, choose the maximum score.
5. If it is the minimizing player's turn, choose the minimum score.
6. Repeat recursively until the game ends.
7. Display the best move and its score.
8. Stop.

Program (Python)

```
# Minimax Algorithm
```

```
scores = {  
    'A': 3,  
    'B': 5,  
    'C': 2,  
    'D': 9  
}
```



```

def minimax(states, is_max):
    if len(states) == 1:
        return scores[states[0]]
    mid = len(states) // 2
    left = minimax(states[:mid], not is_max)
    right = minimax(states[mid:], not is_max)
    if is_max:
        return max(left, right)
    else:
        return min(left, right)

states = ['A', 'B', 'C', 'D']
result = minimax(states, True)
print("Best Score:", result)

```

Sample Input

Game States:

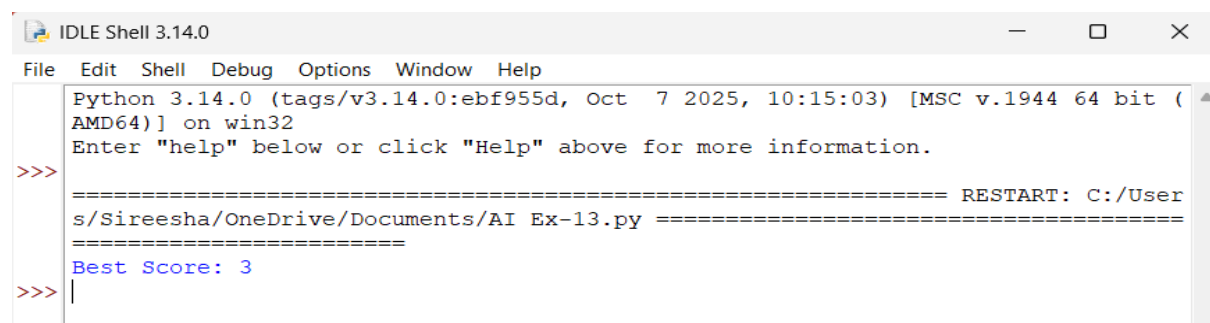
A = 3

B = 5

C = 2

D = 9

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
===== Restart: C:/Users/Sireesha/OneDrive/Documents/AI Ex-13.py =====
Best Score: 3
>>>

```

Result

The Python program for the Minimax Algorithm was successfully implemented. The program evaluated the game states recursively and displayed the best possible score for the maximizing player.

Ex.No-14

Experiment : Write a Python Program to Implement Alpha-Beta Pruning Algorithm for Gaming

Aim

The aim of this experiment is to write a Python program to implement the Alpha-Beta Pruning algorithm for a game and find the best move by reducing unnecessary searches.

Objectives

- To understand the Alpha-Beta Pruning algorithm.
- To write a Python program for Alpha-Beta Pruning.
- To find the best move efficiently.
- To display the optimal score.

Algorithm

1. Start.
2. Define the game tree and leaf node values.
3. Initialize $\alpha = -\infty$ and $\beta = +\infty$.
4. Traverse the game tree using Minimax.
5. Update α and β values.
6. Prune branches when $\alpha \geq \beta$.
7. Display the best score.
8. Stop.

Program (Python)

```
import math
values = [3, 5, 6, 9, 1, 2, 0, -1]
def alphabeta(depth, index, maximizing, alpha, beta):
    if depth == 3:
        return values[index]
    if maximizing:
        best = -math.inf
        for i in range(2):
```

```

        val = alphabeta(depth + 1, index * 2 + i, False, alpha, beta)

        best = max(best, val)

        alpha = max(alpha, best)

        if beta <= alpha:

            break

    return best

else:

    best = math.inf

    for i in range(2):

        val = alphabeta(depth + 1, index * 2 + i, True, alpha, beta)

        best = min(best, val)

        beta = min(beta, best)

        if beta <= alpha:

            break

    return best

result = alphabeta(0, 0, True, -math.inf, math.inf)

print("Optimal Value:", result)

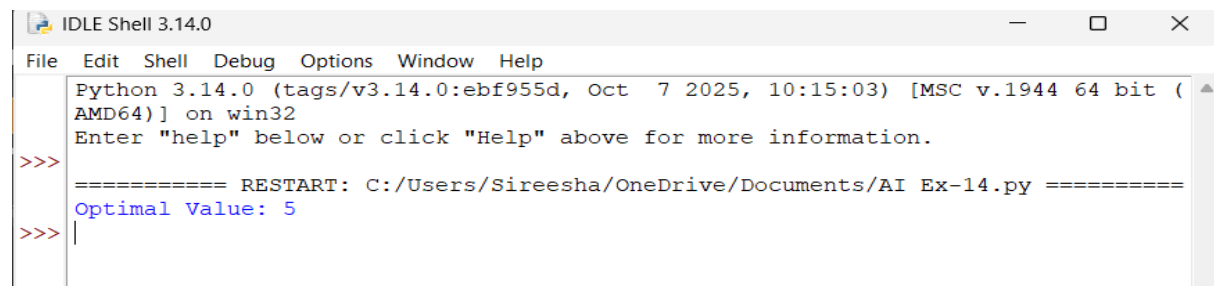
```

Sample Input

Leaf Node Values:

3 5 6 9 1 2 0 -1

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-14.py =====
Optimal Value: 5
>>>

```

Result

The Python program for the Alpha-Beta Pruning Algorithm was successfully implemented. The program efficiently evaluated the game tree by pruning unnecessary branches and displayed the optimal value correctly.

Ex.No-15

Experiment : Write a Python Program to Implement a Decision Tree

Aim

The aim of this experiment is to write a Python program to implement a Decision Tree for classifying data based on the given features.

Objectives

- To understand the Decision Tree algorithm.
- To write a Python program for Decision Tree classification.
- To train the model using sample data.
- To predict the class of new data.

Algorithm

1. Start.
2. Import the required libraries.
3. Prepare the training data and target values.
4. Create a Decision Tree classifier.
5. Train the model using the training data.
6. Predict the class for the test data.
7. Display the prediction.
8. Stop.

Program (Python)

```
# Decision Tree Implementation without sklearn

print("==== Decision Tree Classification =====")
print("Predict whether a person will buy a product")
age = int(input("Enter Age: "))
salary = int(input("Enter Annual Salary: "))
student = input("Are you a Student? (Yes/No): ")
credit = input("Credit Rating (Fair/Excellent): ")
if age < 30:
    if student.lower() == "yes":
```

```

        decision = "Yes"
    else:
        decision = "No"
elif age >= 30 and age <= 40:
    if salary >= 50000:
        if credit.lower() == "fair":
            decision = "Yes"
        else:
            decision = "Yes"
    else:
        decision = "No"
else:
    if credit.lower() == "excellent":
        decision = "No"
    else:
        decision = "Yes"
print("\n----- Decision Tree Result -----")
print("Age      :", age)
print("Salary    :", salary)
print("Student    :", student)
print("Credit Rating :", credit)
print("Prediction  :", decision)

```

Sample Input

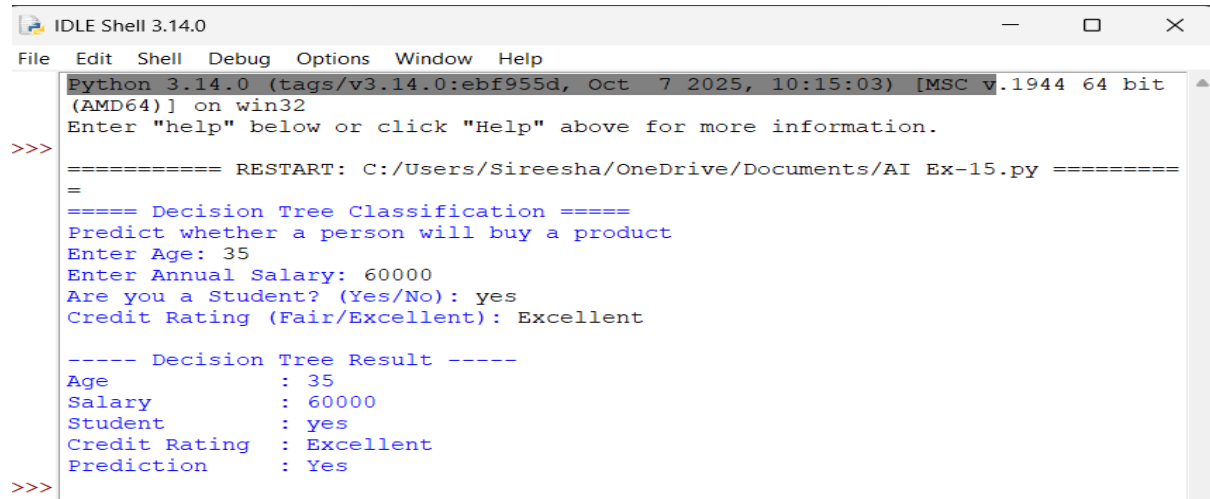
Training Data:

Age	Salary	Buy
25	50000	No
35	65000	Yes
45	80000	Yes
20	30000	No

Test Data:

30 60000

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-15.py =====
===== Decision Tree Classification =====
Predict whether a person will buy a product
Enter Age: 35
Enter Annual Salary: 60000
Are you a Student? (Yes/No): yes
Credit Rating (Fair/Excellent): Excellent

----- Decision Tree Result -----
Age           : 35
Salary        : 60000
Student       : yes
Credit Rating : Excellent
Prediction    : Yes
>>>
```

Result

The Python program for the Decision Tree algorithm was successfully implemented. The model was trained using the given data and correctly predicted the class for the test data.

Experiment : Write a Python Program to Implement a Feed Forward Neural Network

Aim

The aim of this experiment is to write a Python program to implement a Feed Forward Neural Network for processing input data and producing the corresponding output.

Objectives

- To understand the Feed Forward Neural Network.
- To write a Python program for a Feed Forward Neural Network.
- To process input through hidden and output layers.
- To display the output of the network.

Algorithm

1. Start.
2. Initialize the input values, weights, and biases.
3. Calculate the output of the hidden layer.
4. Apply the sigmoid activation function.
5. Calculate the output of the output layer.
6. Apply the sigmoid activation function.
7. Display the final output.
8. Stop.

Program (Python)

```
import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

x1 = 0.5
x2 = 0.8
w1 = 0.2
w2 = 0.4
w3 = 0.3
w4 = 0.7
```

```

w5 = 0.6
w6 = 0.9
b1 = 0.5
b2 = 0.5

h1 = sigmoid(x1 * w1 + x2 * w2 + b1)
h2 = sigmoid(x1 * w3 + x2 * w4 + b1)
output = sigmoid(h1 * w5 + h2 * w6 + b2)
print("Hidden Layer Output:")
print("H1 =", round(h1, 4))
print("H2 =", round(h2, 4))
print("\nFinal Output:")
print(round(output, 4))

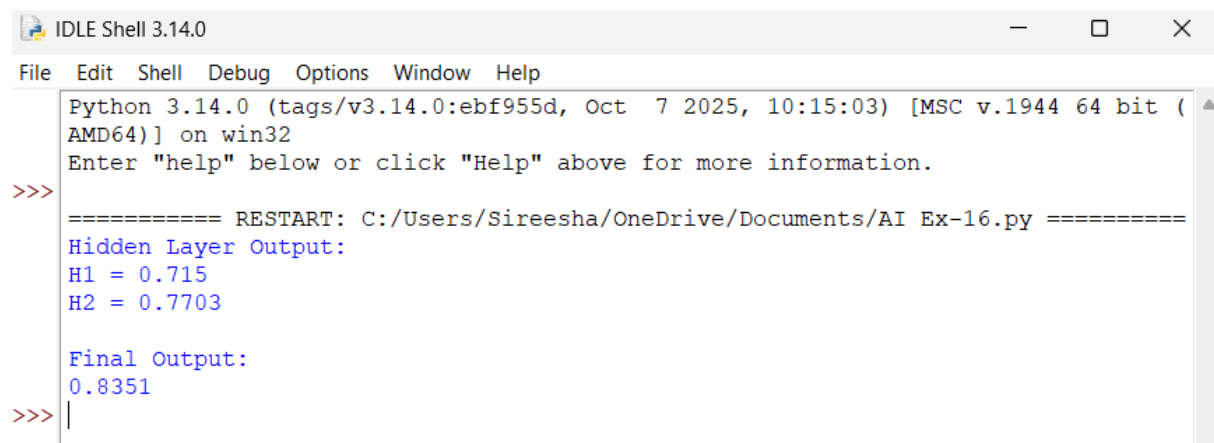
```

Sample Input

Input 1 = 0.5

Input 2 = 0.8

Output



```

IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Sireesha/OneDrive/Documents/AI Ex-16.py =====
Hidden Layer Output:
H1 = 0.715
H2 = 0.7703
Final Output:
0.8351
>>> |

```

Result

The Python program for the Feed Forward Neural Network was successfully implemented. The network processed the input values through the hidden layer and output layer using the sigmoid activation function, and the final output was displayed successfully.

Ex.No:17

Experiment : Write a Prolog Program to Sum the Integers from 1 to n

Aim

To write a Prolog program to find the sum of integers from 1 to n.

Objectives

- To understand recursion in Prolog.
- To calculate the sum of numbers from 1 to n.
- To display the result.

Algorithm

1. Start.
2. Read the value of n.
3. If $n = 0$, return 0.
4. Otherwise, add n to the sum of $n-1$.
5. Display the result.
6. Stop.

Program

```
sum(0,0).
```

```
sum(N,S):-
```

```
    N>0,
```

```
    N1 is N-1,
```

```
    sum(N1,S1),
```

```
    S is N+S1.
```

Sample Input

```
?- sum(5,S).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/sum.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/sum.pl compiled, 5 lines read - 826 bytes written, 8 ms
| ?- sum(5, S).

S = 15 ?
yes
| ?- |
```

Result

Thus, the Prolog program to find the sum of integers from 1 to n was implemented successfully.

Ex.No:18

Experiment : Write a Prolog Program for Name and DOB Database

Aim

To write a Prolog program to create a database containing Name and Date of Birth (DOB).

Objectives

- To understand Prolog facts.
- To create a simple database.
- To retrieve the stored information.

Algorithm

1. Start.
2. Store Name and DOB as facts.
3. Enter a query.
4. Display the matching record.
5. Stop.

Program

```
person(ram,'10-01-2002').  
person(sita,'15-08-2003').  
person(ravi,'25-12-2001').  
person(priya,'05-06-2004').
```

Sample Input

?- person(ram,DOB).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/person DOB.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/person DOB.pl compiled, 3 lines read - 629 bytes written, 9 ms
| ?- person(ram,DOB).

DOB = '10-01-2002'

yes
| ?- |
```

Result

Thus, the Prolog program to create a Name and DOB database was implemented successfully.

Experiment : Write a Prolog Program for Student–Teacher–Subject Code Database

Aim

To write a Prolog program to create a database containing Student Name, Teacher Name, and Subject Code.

Objectives

- To understand Prolog databases.
- To store student details.
- To retrieve teacher and subject information.

Algorithm

1. Start.
2. Store Student, Teacher, and Subject Code.
3. Enter a query.
4. Display the result.
5. Stop.

Program

```
student_teacher(ram,ramesh,cs101).
```

```
student_teacher(sita,suresh,ai102).
```

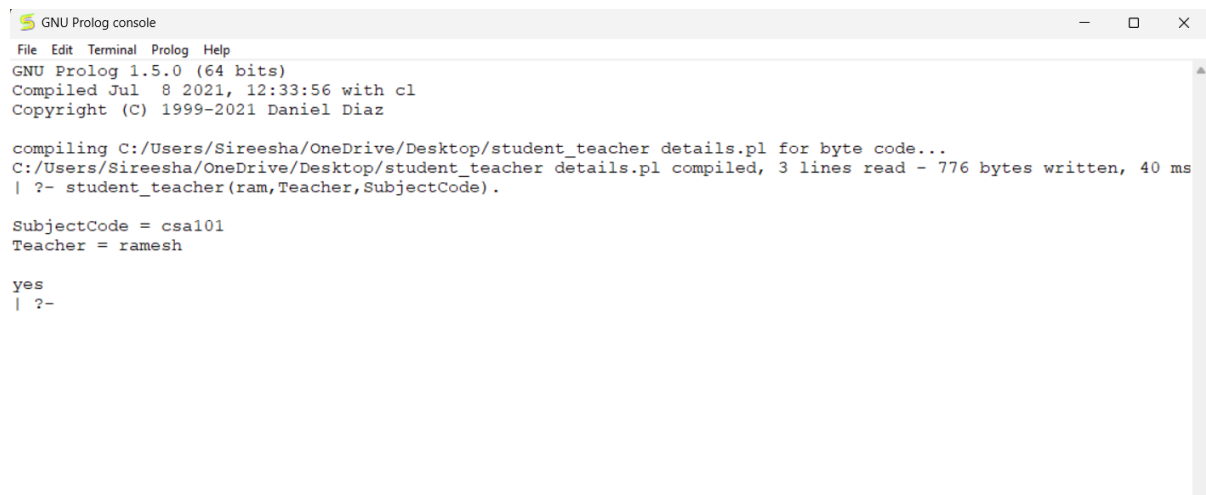
```
student_teacher(ravi,mahesh,db103).
```

```
student_teacher(priya,anitha,cn104).
```

Sample Input

```
?- student_teacher(ram,Teacher,SubjectCode).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/student_teacher details.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/student_teacher details.pl compiled, 3 lines read - 776 bytes written, 40 ms
| ?- student_teacher(ram,Teacher,SubjectCode).

SubjectCode = csal01
Teacher = ramesh

yes
| ?-
```

Result

Thus, the Prolog program for the Student–Teacher–Subject Code database was implemented successfully.

Ex.No:20

Experiment : Write a Prolog Program for Planets Database

Aim

To write a Prolog program to create a database containing the names of planets and their order from the Sun.

Objectives

- To understand Prolog facts.
- To create a planets database.
- To retrieve planet information.

Algorithm

1. Start.
2. Store planet names and positions.
3. Enter a query.
4. Display the required information.
5. Stop.

Program

```
planet(mercury,1).
```

```
planet(venus,2).
```

```
planet(earth,3).
```

```
planet(mars,4).
```

```
planet(jupiter,5).
```

```
planet(saturn,6).
```

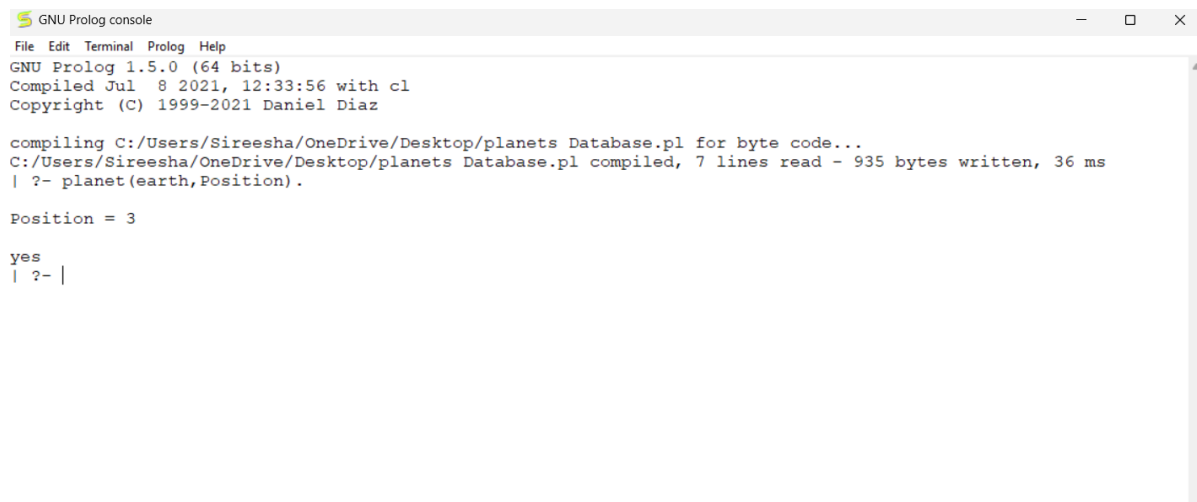
```
planet(uranus,7).
```

```
planet(neptune,8).
```

Sample Input

```
?- planet(earth,Position).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/planets Database.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/planets Database.pl compiled, 7 lines read - 935 bytes written, 36 ms
| ?- planet(earth,Position).

Position = 3

yes
| ?- |
```

Result

Thus, the Prolog program for the Planets Database was implemented successfully.

Ex.No:21

Experiment : Write a Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to solve the Towers of Hanoi problem.

Objectives

- To understand recursion in Prolog.
- To move all disks from source to destination.
- To follow the rules of the Towers of Hanoi.

Algorithm

1. Start.
2. Read the number of disks.
3. Move N-1 disks from source to auxiliary.
4. Move the largest disk to destination.
5. Move N-1 disks from auxiliary to destination.
6. Stop.

Program

```
move(1,X,Y,_):-
```

```
    write('Move disk from '),
```

```
    write(X),
```

```
    write(' to '),
```

```
    write(Y),nl.
```

```
move(N,X,Y,Z):-
```

```
    N>1,
```

```
    N1 is N-1,
```

```
    move(N1,X,Z,Y),
```

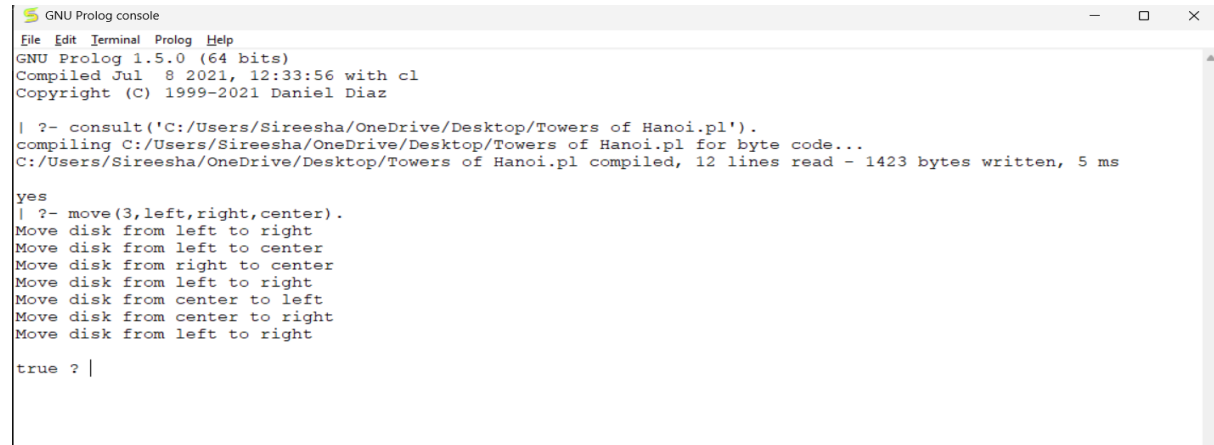
```
    move(1,X,Y,_),
```

```
    move(N1,Z,Y,X).
```


Sample Input

?- move(3,left,right,center).

Output

A screenshot of a GNU Prolog console window. The window has a title bar with the text "GNU Prolog console" and standard window controls. The menu bar includes "File", "Edit", "Terminal", "Prolog", and "Help". The console output shows the Prolog version (1.5.0), compilation details (compiled Jul 8 2021), and the execution of a Towers of Hanoi program. The user enters the query "?- consult('C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl').", which is followed by compilation messages. Then, the user enters "?- move(3,left,right,center).", and the program outputs a sequence of seven moves. Finally, the user enters "true ? |", and the console shows a blank line.

```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul 8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl').
compiling C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl compiled, 12 lines read - 1423 bytes written, 5 ms

yes
| ?- move(3,left,right,center).
Move disk from left to right
Move disk from left to center
Move disk from right to center
Move disk from left to right
Move disk from center to left
Move disk from center to right
Move disk from left to right

true ? |
```

Result

Thus, the Towers of Hanoi problem was implemented successfully using Prolog.

Ex.No:22

Experiment 22: Write a Prolog Program to Print Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a bird can fly.

Objectives

- To understand facts and rules.
- To identify birds that can fly.
- To display the result.

Algorithm

1. Start.
2. Store bird names.
3. Define birds that can fly.
4. Query the bird.
5. Display whether it can fly.
6. Stop.

Program

bird(parrot).

bird(crow).

bird(pigeon).

bird(peacock).

bird(ostrich).

bird(penguin).

can_fly(parrot).

can_fly(crow).

can_fly(pigeon).

can_fly(peacock).

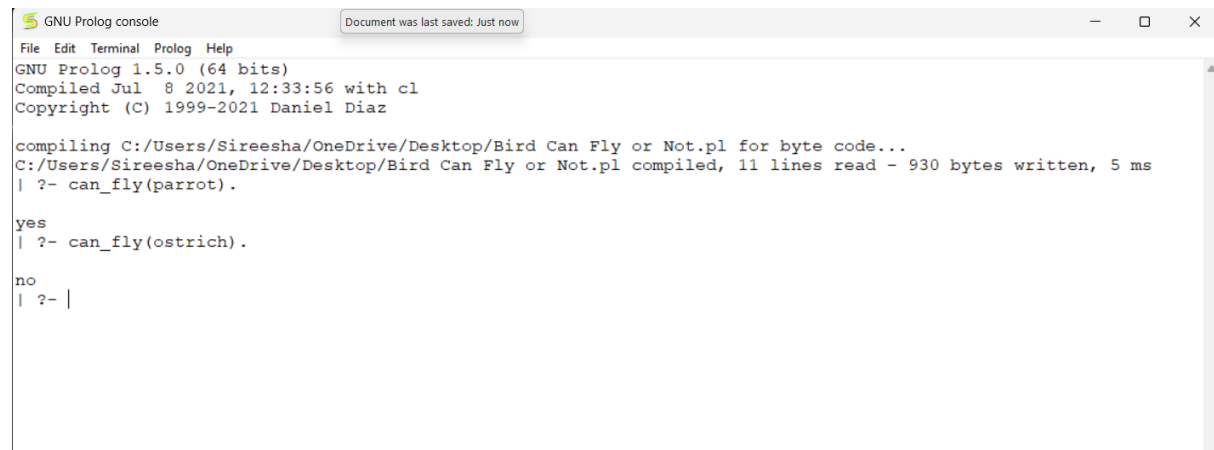
Sample Input

?- can_fly(parrot).

Another Input

?- can_fly(ostrich).

Output

A screenshot of the GNU Prolog console window. The window has a title bar with the text "GNU Prolog console" and a status bar that says "Document was last saved: Just now". The menu bar includes "File", "Edit", "Terminal", "Prolog", and "Help". The main text area shows the following output:

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Bird Can Fly or Not.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Bird Can Fly or Not.pl compiled, 11 lines read - 930 bytes written, 5 ms
| ?- can_fly(parrot).

yes
| ?- can_fly(ostrich).

no
| ?- |
```

Result

Thus, the Prolog program to determine whether a bird can fly was implemented successfully.

Ex.No:23

Experiment 23: Write a Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree.

Objectives

- To understand family relationships.
- To implement parent relationships.
- To retrieve family information.

Algorithm

1. Start.
2. Store parent relationships.
3. Define father and mother.
4. Query the relationships.
5. Display the result.
6. Stop.

Program

```
father(ramesh,ram).
```

```
father(ramesh,sita).
```

```
mother(latha,ram).
```

```
mother(latha,sita).
```

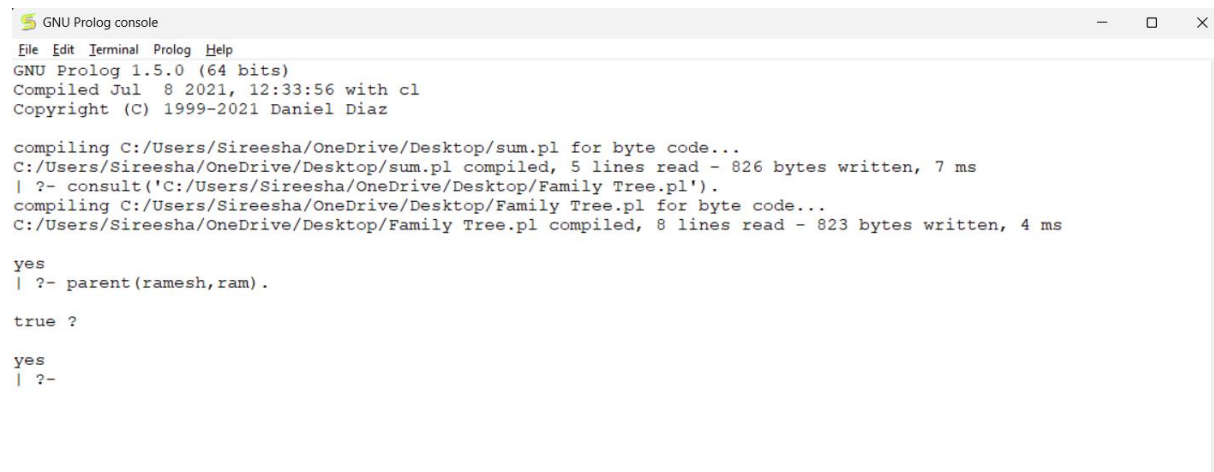
```
parent(X,Y):-father(X,Y).
```

```
parent(X,Y):-mother(X,Y).
```

Sample Input

```
?- parent(ramesh,ram).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/sum.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/sum.pl compiled, 5 lines read - 826 bytes written, 7 ms
| ?- consult('C:/Users/Sireesha/OneDrive/Desktop/Family Tree.pl').
compiling C:/Users/Sireesha/OneDrive/Desktop/Family Tree.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Family Tree.pl compiled, 8 lines read - 823 bytes written, 4 ms

yes
| ?- parent(ramesh,ram).

true ?

yes
| ?-
```

Result

Thus, the family tree was implemented successfully using Prolog.

Ex.No:24

Experiment : Write a Prolog Program to Suggest Dieting System Based on Disease

Aim

To write a Prolog program to suggest a suitable diet based on the disease.

Objectives

- To understand Prolog facts.
- To store disease and diet information.
- To suggest an appropriate diet.

Algorithm

1. Start.
2. Store diseases and diets.
3. Enter the disease.
4. Display the recommended diet.
5. Stop.

Program

```
diet(diabetes,'Low Sugar Diet').
```

```
diet(bp,'Low Salt Diet').
```

```
diet(obesity,'Low Fat Diet').
```

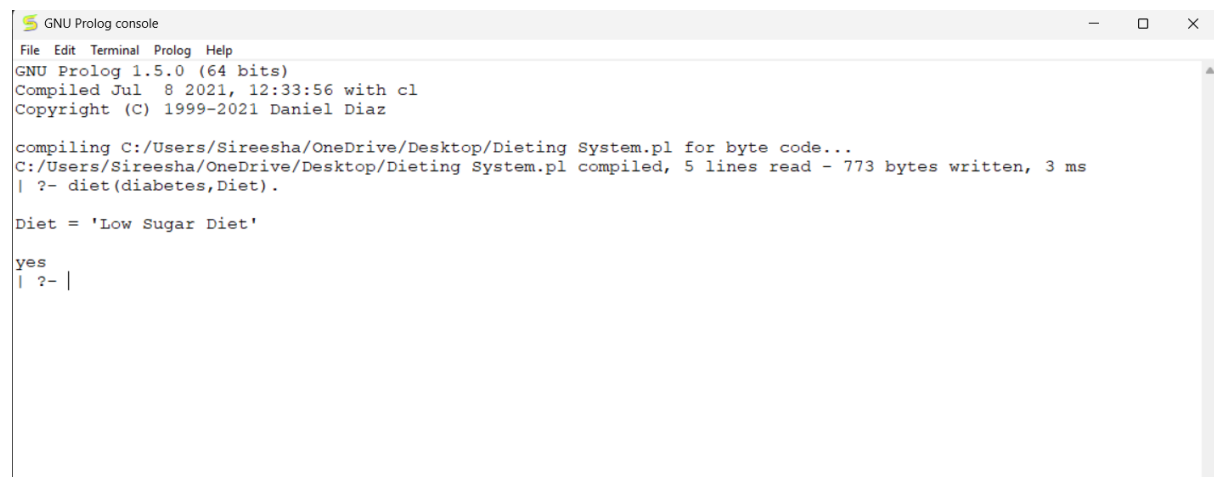
```
diet(fever,'Liquid Diet').
```

```
diet(anemia,'Iron Rich Diet').
```

Sample Input

?- diet(diabetes,Diet).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Dieting System.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Dieting System.pl compiled, 5 lines read - 773 bytes written, 3 ms
| ?- diet(diabetes,Diet).

Diet = 'Low Sugar Diet'

yes
| ?- |
```

Result

Thus, the Prolog program to suggest a diet based on disease was implemented successfully.

Ex.No:25

Experiment : Write a Prolog Program to Implement Monkey Banana Problem

Aim

To write a Prolog program to solve the Monkey Banana problem.

Objectives

- To understand the Monkey Banana problem.
- To represent actions using Prolog.
- To determine whether the monkey can reach the banana.

Algorithm

1. Start.
2. Define the monkey's initial position.
3. Define the box position.
4. Move the monkey to the box.
5. Push the box under the banana.
6. Climb the box.
7. Grab the banana.
8. Stop.

Program

at(monkey,door).

at(box>window).

at(banana,center).

move(monkey,door>window).

push(box>window,center).

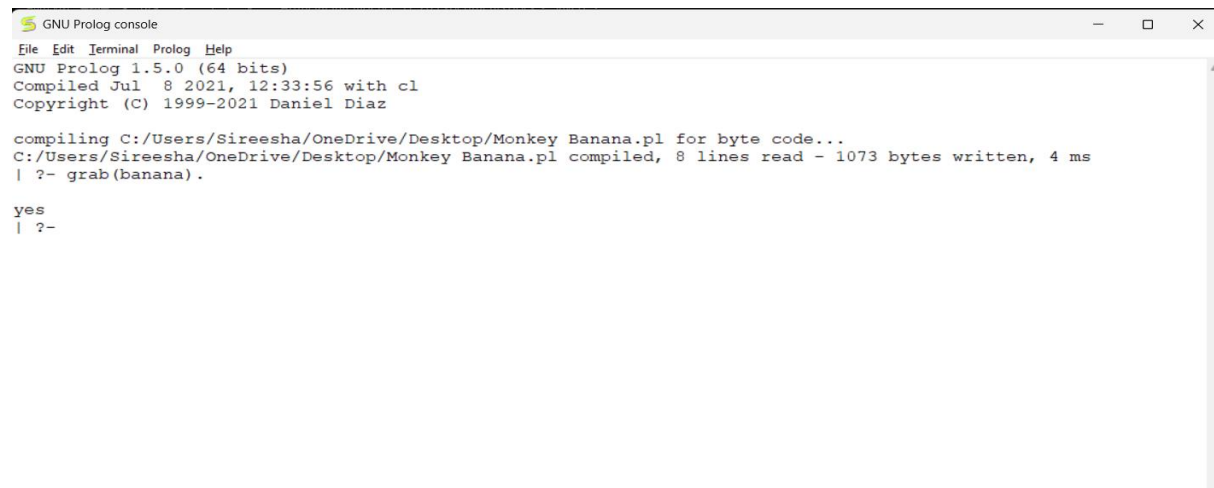
climb(monkey).

grab(banana).

Sample Input

?- grab(banana).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Monkey Banana.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Monkey Banana.pl compiled, 8 lines read - 1073 bytes written, 4 ms
| ?- grab(banana).

yes
| ?-
```

Result

Thus, the Monkey Banana problem was implemented successfully using Prolog.

Ex.No:26

Experiment 26: Write a Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write a Prolog program to store the names of fruits and their colors and retrieve the information using backtracking.

Objectives

- To understand backtracking in Prolog.
- To create a fruit and color database.
- To retrieve fruit names based on color.
- To display all possible solutions using backtracking.

Algorithm

1. Start.
2. Store fruits and their corresponding colors as facts.
3. Enter a query with a fruit name or color.
4. Prolog searches the database.
5. If multiple solutions exist, use backtracking to display all of them.
6. Stop.

Program (Prolog)

```
% Fruit and Color Database
```

```
fruit(apple, red).
```

```
fruit(banana, yellow).
```

```
fruit(grapes, green).
```

```
fruit(mango, yellow).
```

```
fruit(orange, orange).
```

```
fruit(guava, green).
```

Sample Input 1

```
?- fruit(apple, Color).
```

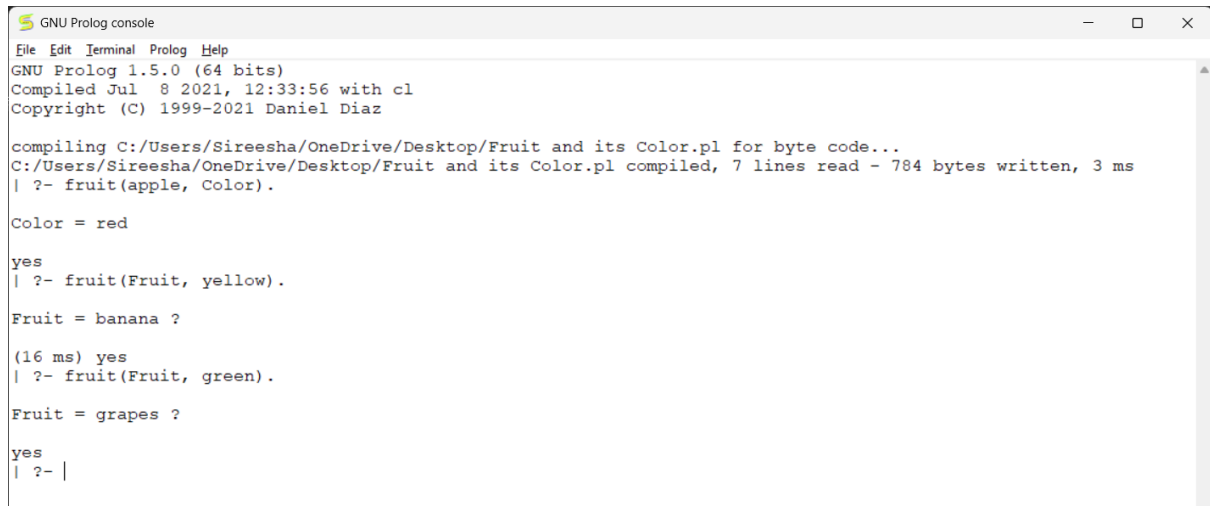
Sample Input 2

?- fruit(Fruit, yellow).

Sample Input 3

?- fruit(Fruit, green).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Fruit and its Color.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Fruit and its Color.pl compiled, 7 lines read - 784 bytes written, 3 ms
| ?- fruit(apple, Color).

Color = red

yes
| ?- fruit(Fruit, yellow).

Fruit = banana ?

(16 ms) yes
| ?- fruit(Fruit, green).

Fruit = grapes ?

yes
| ?- |
```

Result

Thus, the Prolog program for Fruit and Its Color Using Backtracking was successfully implemented. The program retrieved all matching fruits for a given color using Prolog's backtracking mechanism.

Ex.No:27

Experiment : Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm and find a path from a starting node to a goal node using heuristic values.

Objectives

- To understand the Best First Search algorithm.
- To use heuristic values for searching.
- To find a path from the start node to the goal node.
- To implement the algorithm using Prolog.

Algorithm

1. Start.
2. Define the graph and heuristic values.
3. Set the starting node.
4. Select the node with the smallest heuristic value.
5. Check whether it is the goal node.
6. If it is not the goal, expand its neighboring nodes.
7. Add the neighboring nodes to the search list.
8. Select the node with the lowest heuristic value.
9. Repeat until the goal is reached.
10. Display the path.
11. Stop.

Program

% Best First Search

edge(a, b).

edge(a, c).

edge(b, d).

edge(b, e).

edge(c, f).

edge(c, g).

edge(d, h).

edge(e, h).

edge(f, h).

edge(g, h).

heuristic(a, 7).

heuristic(b, 6).

heuristic(c, 5).

heuristic(d, 4).

heuristic(e, 3).

heuristic(f, 2).

heuristic(g, 1).

heuristic(h, 0).

best_first(Start, Goal, Path) :-

 search([Start], Goal, [], RevPath),

 reverse(RevPath, Path).

search([Goal|_], Goal, Visited, [Goal|Visited]).

search(Open, Goal, Visited, Path) :-

 Open = [Current|Rest],

 findall(H-Next,

 (edge(Current, Next),

 \+ member(Next, Visited),

 heuristic(Next, H)),

```

        Children),
    add_children(Children, Rest, NewOpen),
    search(NewOpen, Goal, [Current|Visited], Path).
add_children([], Open, Open).
add_children([H-Node|Rest], Open, NewOpen) :-
    insert_by_heuristic(H-Node, Open, Temp),
    add_children(Rest, Temp, NewOpen).
insert_by_heuristic(H-Node, [], [Node]) :-
    heuristic(Node, H).
insert_by_heuristic(H-Node, [First|Rest], [Node,First|Rest]) :-
    heuristic(First, H1),
    H =< H1.
insert_by_heuristic(H-Node, [First|Rest], [First|NewRest]) :-
    insert_by_heuristic(H-Node, Rest, NewRest).

```

Sample Input

```
?- best_first(a, h, Path).
```

Sample Output

```

GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul 8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Best First Search Algorithm.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Best First Search Algorithm.pl for byte code...
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Best First Search Algorithm.pl compiled, 52 lines read

yes
| ?- best_first(a, h, Path).

Path = [a,c,g,h] ?
yes
| ?- |

```

Result

Thus, the Best First Search algorithm was successfully implemented in Prolog, and a path from the starting node a to the goal node h was obtained using heuristic values.

Ex.No:28

Experiment : Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to identify a possible disease based on the symptoms given by the user.

Objectives

- To understand facts and rules in Prolog.
- To represent symptoms and diseases.
- To diagnose a disease based on given symptoms.
- To use Prolog queries for diagnosis.

Algorithm

1. Start.
2. Define symptoms for different diseases.
3. Define rules for diagnosing diseases.
4. Enter the symptoms as a query.
5. Prolog checks the matching rules.
6. Display the possible disease.
7. Stop.

Program

```
% Medical Diagnosis System
```

```
symptom(fever).
```

```
symptom(cough).
```

```
symptom(cold).
```

```
symptom(headache).
```

```
symptom(body_pain).
```

```
symptom(sneezing).
```

symptom(sore_throat).

disease(flu) :-

symptom(fever),

symptom(cough),

symptom(body_pain).

disease(common_cold) :-

symptom(cold),

symptom(sneezing),

symptom(sore_throat).

disease(covid) :-

symptom(fever),

symptom(cough),

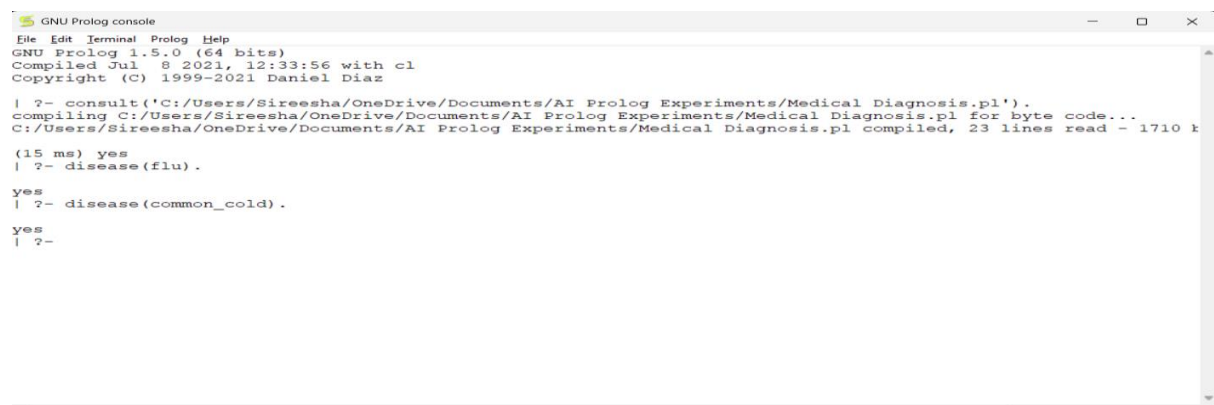
symptom(sore_throat).

Sample Input

?- disease(flu).

?-disease(common_cold).

Sample Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Medical Diagnosis.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Medical Diagnosis.pl for byte code...
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Medical Diagnosis.pl compiled, 23 lines read - 1710 b
(15 ms) yes
| ?- disease(flu) .

yes
| ?- disease(common_cold) .

yes
| ?-
```

Result

Thus, the Prolog program for Medical Diagnosis was successfully implemented using symptoms and rules.

Ex.No:29

Experiment 29: Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement **Forward Chaining** and derive a conclusion from known facts and rules.

Objectives

- To understand Forward Chaining.
- To define facts and rules.
- To derive new facts from existing facts.
- To perform the required queries.

Algorithm

1. Start.
2. Define the initial facts.
3. Define rules based on the facts.
4. Apply the rules to derive new facts.
5. Continue until no new facts can be derived.
6. Query the required conclusion.
7. Display the result.
8. Stop.

Program

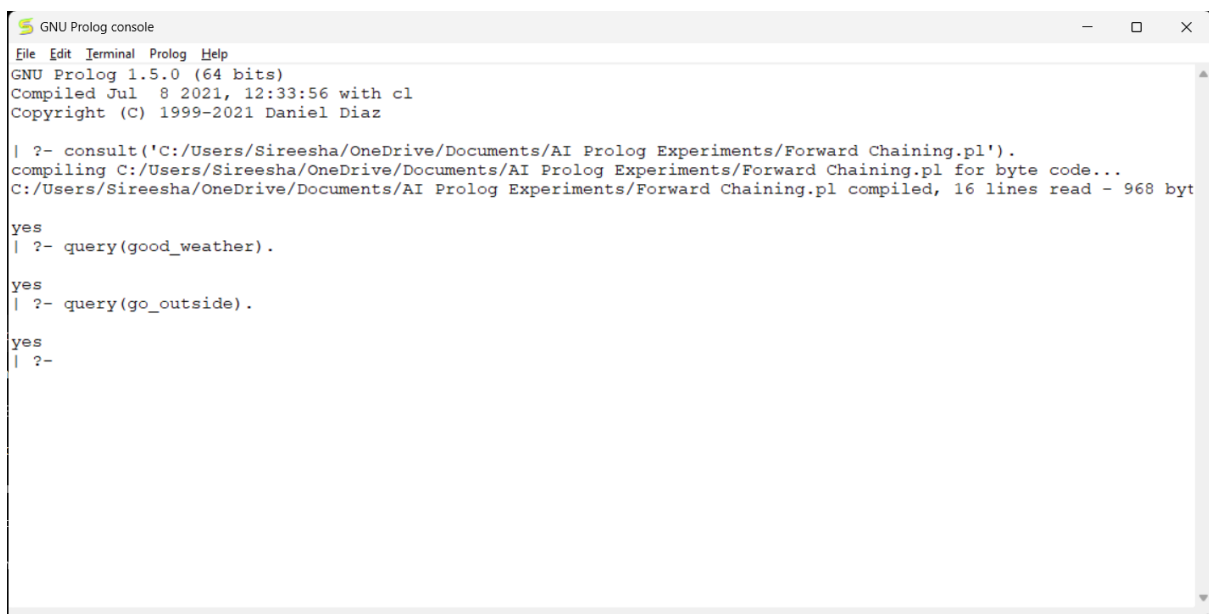
```
% Forward Chaining  
fact(sunny).  
fact(warm).  
rule(good_weather) :-  
    fact(sunny),  
    fact(warm).
```

```
rule(go_outside) :-  
    rule(good_weather).  
  
query(X) :-  
    fact(X).  
  
query(X) :-  
    rule(X).
```

Sample Input

```
?- query(good_weather).  
  
?- query(go_outside).
```

Sample Output



```
GNU Prolog console  
File Edit Terminal Prolog Help  
GNU Prolog 1.5.0 (64 bits)  
Compiled Jul  8 2021, 12:33:56 with cl  
Copyright (C) 1999-2021 Daniel Diaz  
  
| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Forward Chaining.pl').  
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Forward Chaining.pl for byte code...  
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Forward Chaining.pl compiled, 16 lines read - 968 byt  
  
yes  
| ?- query(good_weather).  
  
yes  
| ?- query(go_outside).  
  
yes  
| ?-
```

Result

Thus, the Prolog program for Forward Chaining was successfully implemented and the required conclusions were derived from the given facts and rules.

Ex.No:30

Experiment : Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining by proving a goal using available facts and rules.

Objectives

- To understand Backward Chaining.
- To define facts and rules.
- To prove a given goal.
- To perform the required queries.

Algorithm

1. Start.
2. Define facts and rules and Select a goal to prove.
3. Check whether the goal is a known fact.
4. If not, find a rule that can prove the goal.
5. Prove the conditions of that rule.
6. If all conditions are true, the goal is true.
7. Display the result.
8. Stop.

Program

```
% Backward Chaining
```

```
bird(parrot).
```

```
has_feathers(parrot).
```

```
can_fly(X) :-
```

```
    bird(X),
```

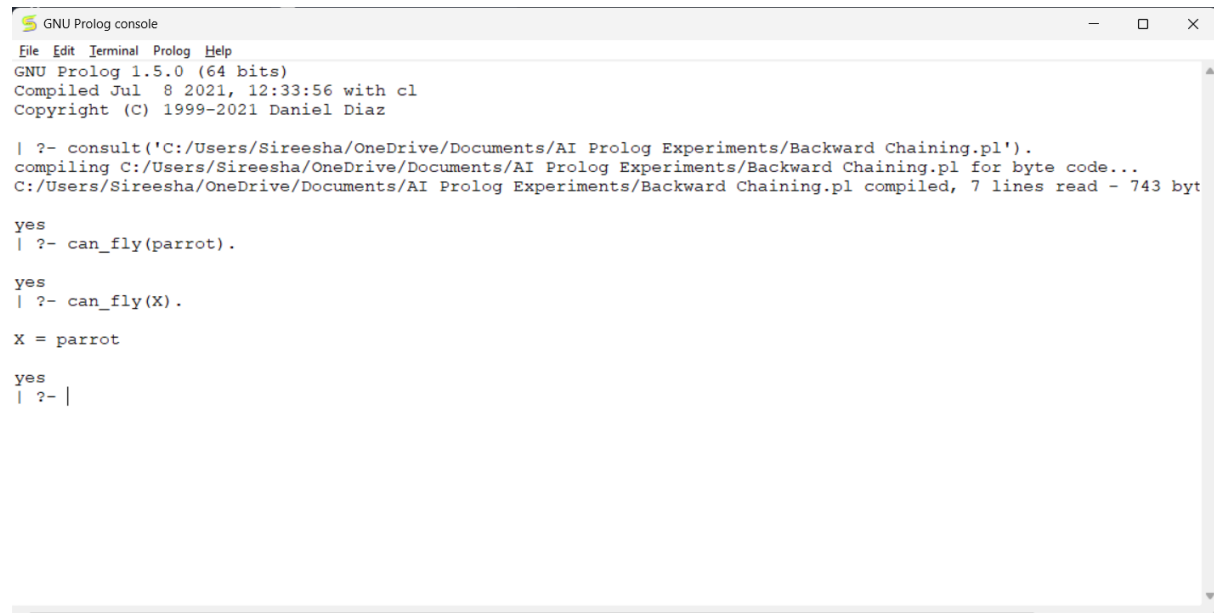
```
    has_feathers(X).
```

Sample Input

?- can_fly(parrot).

?- can_fly(X).

Sample Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Backward Chaining.pl').
compiling C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Backward Chaining.pl for byte code...
C:/Users/Sireesha/OneDrive/Documents/AI Prolog Experiments/Backward Chaining.pl compiled, 7 lines read - 743 bytes

yes
| ?- can_fly(parrot).

yes
| ?- can_fly(X).

X = parrot

yes
| ?- |
```

Result

Thus, the Prolog program for Backward Chaining was successfully implemented, and the given goal was proved using facts and rules.

Ex.No:31

Experiment : Create a Web Blog Using WordPress to Demonstrate Anchor Tag, Title Tag, etc.

Aim

To create a simple web blog using WordPress and demonstrate HTML elements such as the Title Tag, Anchor Tag, Heading Tag, Paragraph Tag, and Image.

Objectives

- To understand the basic features of WordPress.
- To create and publish a simple blog.
- To demonstrate the use of HTML tags.

- To add hyperlinks using the Anchor Tag.
- To use title and heading elements.

Algorithm

1. Start.
2. Open WordPress and create a new website/blog.
3. Create a new blog post.
4. Add a suitable title.
5. Add headings and paragraphs.
6. Add an image to the blog.
7. Add a hyperlink using the Anchor Tag.
8. Preview the blog.
9. Publish the blog.
10. Stop.

Program / HTML Code

The following HTML can be used in a Custom HTML block in WordPress:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>My Educational Blog</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Welcome to My Educational Blog</h1>
```

```
    <h2>About My Blog</h2>
```

<p>

This blog provides useful information about
computer science and technology.

</p>

<h2>Useful Website</h2>

<p>

Visit

WordPress

to create your own blog.

</p>

<h2>My Topics</h2>

<p>

Python, Artificial Intelligence, Machine Learning
and Web Development.

</p>

</body>

</html>

Sample Input

Blog Title: My Educational Blog

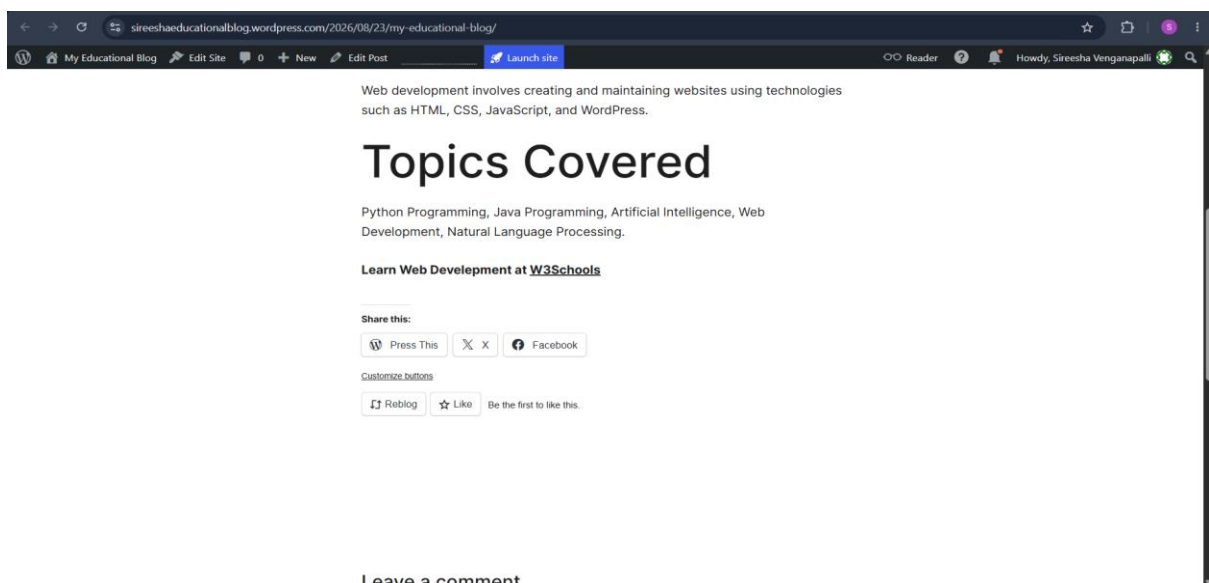
Heading: Welcome to My Educational Blog

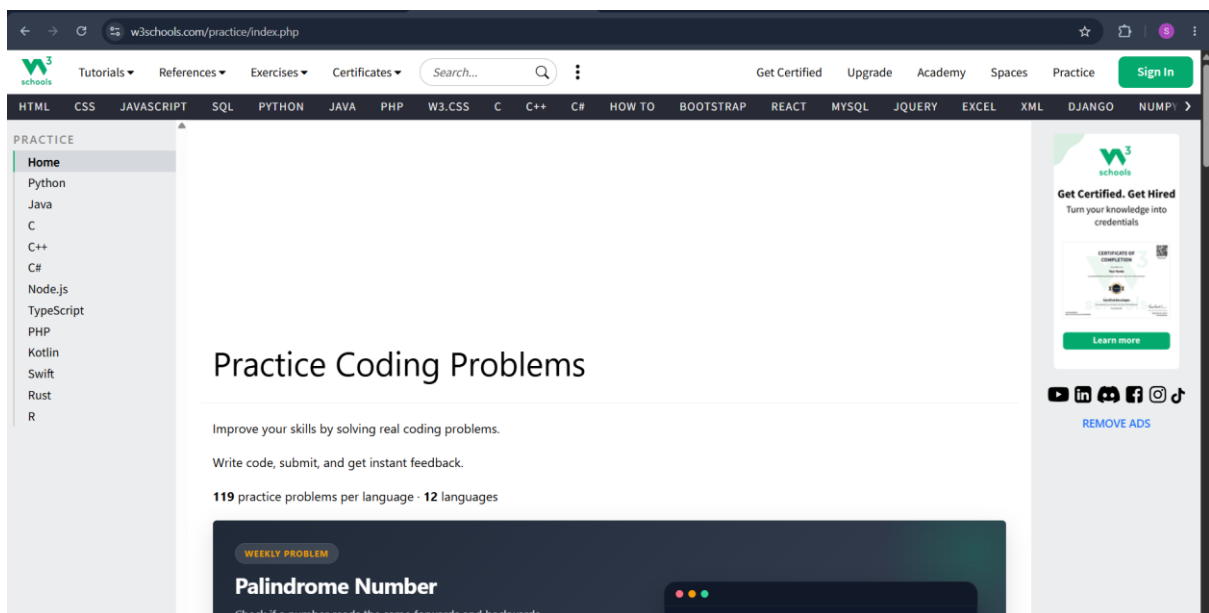
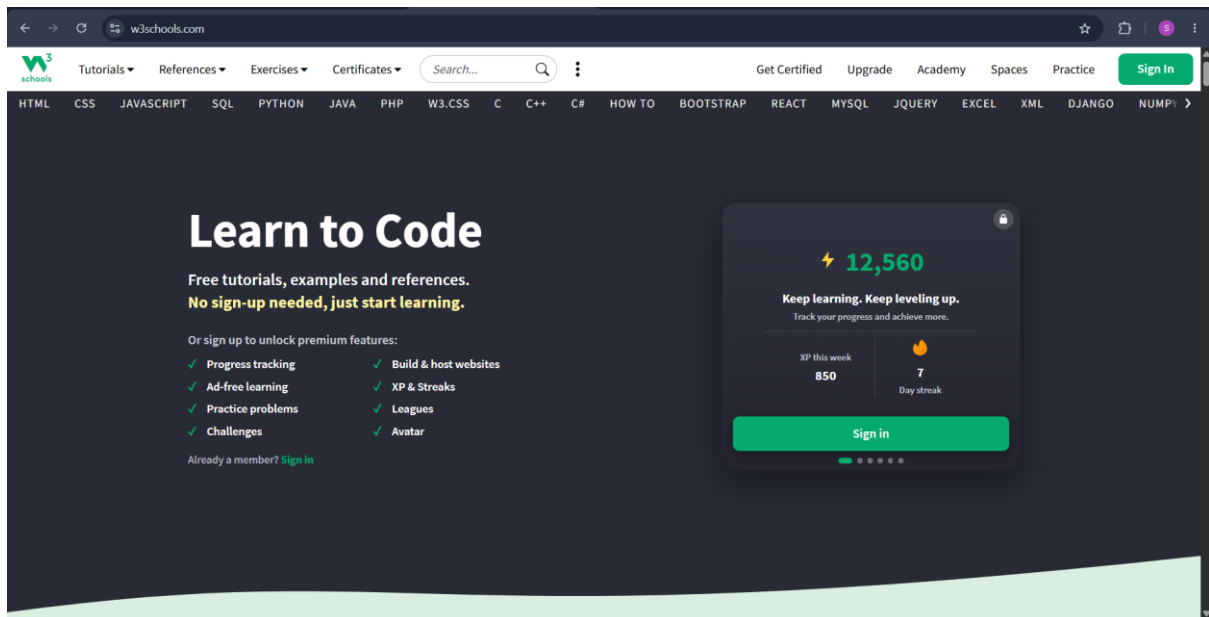
Content:

This blog provides useful information about computer science and technology.

Anchor Text: WordPress

Sample Output





Result

Thus, a simple web blog using WordPress was successfully created, demonstrating the Title Tag, Heading Tags, Paragraph Tag, Anchor Tag, and other basic HTML elements.